ASSIGNMENT-2.5

R.SATHWIK

2303A52T01

QUESTION:

1	Lab 2: Exploring Additional AI Coding Tools beyond Copilot – Gemini (Colab) and Cursor AI **Lab Objectives:** ❖ To explore and evaluate the functionality of Google Gemini for AI-assisted coding within Google Colab. ❖ To understand and use Cursor AI for code generation, explanation, and refactoring. ❖ To compare outputs and usability between Gemini, GitHub Copilot, and Cursor AI. ❖ To perform code optimization and documentation using AI tools. **Lab Outcomes (LOs):** After completing this lab, students will be able to: ❖ Generate Python code using Google Gemini in Google Colab. ❖ Analyze the effectiveness of code explanations and suggestions by Gemini. ❖ Set up and use Cursor AI for AI-powered coding assistance. ❖ Evaluate and refactor code using Cursor AI features. ❖ Compare AI tool behavior and code quality across different platforms.	Week1 - Monday
	Task 1: Refactoring Odd/Even Logic (List Version)	

- **Scenario:**
  You are improving legacy code.

- **Task:**
  Write a program to calculate the sum of odd and even numbers in a list, then refactor it using AI.

- **Expected Output:**
- Original and improved code

Task 2: Area Calculation Explanation

- **Scenario:**
  You are onboarding a junior developer.

- **Task:**
  Ask Gemini to explain a function that calculates the area of different shapes.

- **Expected Output:**
  - Code
  - Explanation

Task 3: Prompt Sensitivity Experiment

- **Scenario:**
  You are testing how AI responds to different prompts.

	❖ **Task:** Use Cursor AI with different prompts for the same problem and observe code changes. ❖ **Expected Output:** ➢ **Prompt list** ➢ **Code variations**	
	Task 4: Tool Comparison Reflection ❖ **Scenario:** You must recommend an AI coding tool. ❖ **Task:** Based on your work in this topic, compare Gemini, Copilot, and Cursor AI for usability and code quality. ❖ **Expected Output:** Short written reflection **Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.**	

CODE: """Task 1: Refactoring Odd/Even Logic (List Version)

❖ Scenario:

You are improving legacy code.

❖ Task:

Write a program to calculate the sum of odd and even numbers in a list,

then refactor it using AI.

❖ Expected Output:

❖ Original and improved code

```
# Task 1: Odd/Even Sum with User Input
# Take list input from user
numbers = list(map(int, input("Enter numbers separated by space: ").split()))
# --- Original Logic ---
even_sum = 0
odd_sum = 0
for num in numbers:
    if num % 2 == 0:
        even_sum += num
    else:
        odd_sum += num
print("\nOriginal Method")
print("Even Sum:", even_sum)
print("Odd Sum:", odd_sum)

# --- Improved Pythonic Logic ---
even_sum_new = sum(num for num in numbers if num % 2 == 0)
odd_sum_new = sum(num for num in numbers if num % 2 != 0)
print("\nImproved Method")
print("Even Sum:", even_sum_new)
print("Odd Sum:", odd_sum_new)"""


"""Task 2: Area Calculation Explanation
```

❖ Scenario:

You are onboarding a junior developer.

❖ Task:

Ask Gemini to explain a function that calculates the area of different shapes.

❖ Expected Output:

➢ Code

➢ Explanation

```
# Task 2: Area Calculation with User Input
import math
def calculate_area(shape, value1, value2=0):
  if shape == "circle":
    return math.pi * value1 * value1
  elif shape == "rectangle":
    return value1 * value2
  elif shape == "triangle":
    return 0.5 * value1 * value2
  else:
    return "Invalid Shape"
shape = input("Enter shape (circle/rectangle/triangle): ").lower()
if shape == "circle":
  r = float(input("Enter radius: "))
  print("Area:", calculate_area(shape, r))
elif shape in ["rectangle", "triangle"]:
  v1 = float(input("Enter value1: "))
  v2 = float(input("Enter value2: "))
```

```
    print("Area:", calculate_area(shape, v1, v2))
else:
    print("Invalid Shape")"""

"""Task 3: Prompt Sensitivity Experiment
```

❖ Scenario:

You are testing how AI responds to different prompts.

❖ Task:

Use Cursor AI with different prompts for the same problem and observe code changes.

❖ Expected Output:

➢ Prompt list

➢ Code variations

```
# Task 3: String Reversal Variations with User Input
text = input("Enter string: ")

# Simple Version
print("\nSimple Version:", text[::-1])

# Function Version
def reverse_string(s):
    return s[::-1]
print("Function Version:", reverse_string(text))

# Commented Version
def reverse_string_commented(text):
```

```
    # Reverse string using slicing
    return text[::-1]
print("Commented Version:", reverse_string_commented(text))"""

"""Task 4: Tool Comparison Reflection
❖ Scenario:
You must recommend an AI coding tool.
❖ Task:
Based on your work in this topic, compare Gemini, Copilot, and Cursor AI
for usability and code quality.
❖ Expected Output:
Short written reflection"""

# Task 4: AI Tool Comparison with User Input

tools = {
    "copilot": {
        "Name": "GitHub Copilot",
        "Usability": "Very Easy (IDE Integrated)",
        "Code Quality": "High"
    },
    "gemini": {
        "Name": "Gemini",
        "Usability": "Easy (Chat Based)",
        "Code Quality": "Good"
    },
    "cursor": {
        "Name": "Cursor AI",
```

```
        "Usability": "Moderate",
        "Code Quality": "Good"
    }
}

choice = input("Enter tool (copilot/gemini/cursor): ").lower()

if choice in tools:
    print("\nTool Details:")
    for key, value in tools[choice].items():
        print(f"{key}: {value}")
else:
    print("Invalid Tool Choice")
```

OUTPUT:

```
PS C:\Users\ragis\OneDrive\Desktop\ai assistent lab> & C:\Users\ragis\AppData\Local\Python\pythonc
ore-3.14-64\python.exe "c:/Users/ragis/OneDrive/Desktop/ai assistent lab/ai-coding-1.5 (1).py"
Loop-Based Reversal: !dlroW ,olleH
Slicing-Based Reversal: !dlroW ,olleH
PS C:\Users\ragis\OneDrive\Desktop\ai assistent lab> & C:\Users\ragis\AppData\Local\Python\pythonc
ore-3.14-64\python.exe "c:/Users/ragis/OneDrive/Desktop/ai assistent lab/ai-coding-2.5 (1).py"
Enter tool (copilot/gemini/cursor):
```